MySocial: Estrutura do Projeto

Clone simplificado do X (Twitter), feito em Kotlin + Jetpack Compose, para a disciplina de Desenvolvimento de Aplicativos Móveis.

Documento de apoio para a defesa de código em equipe. Explica como o projeto está organizado e por quê.

Sumário

1. Visão geral do app
2. Separação dos pacotes (pastas)
3. O padrão MVVM aplicado aqui
4. Fluxo completo de uma ação (exemplo: criar um post)
5. Banco de dados e migração automática
6. Decisões de segurança
7. Mapa de telas e rotas

1. Visão geral do app

O MySocial é um app de rede social no estilo do X: qualquer pessoa cadastrada pode publicar posts de até 280 caracteres, seguir outras pessoas e ver um feed com os posts mais recentes.

Funcionalidades implementadas:

- Cadastro e login (com e-mail ou @username) com senha protegida por hash (nunca texto puro).
- Controle de sessão (quem está logado agora).
- Feed de posts, mais recente primeiro.
- Criar, editar e excluir post (CRUD completo), com compositor estilo X (campo grande, sem caixa, "X" pra fechar).
- Perfil de usuário com bio, endereço e contagem de seguidores e seguindo.
- Editar o próprio perfil (bio e endereço: CEP, rua, número, bairro, cidade, estado).
- Busca de usuários por username, pra achar alguém e seguir.
- Seguir e deixar de seguir outro usuário.

- Lista de seguidores e de quem o usuário segue, com abas pra trocar entre as duas (igual ao X).
- Tela de detalhe de um post, com curtida e comentários.
- Curtir e descurtir post.
- Comentar um post e excluir o próprio comentário.
- Perfil com capa colorida (cor própria de cada usuário) e avatar.

2. Separação dos pacotes (pastas)

Todo o código fica em `com.mysocial`, dividido por responsabilidade:

auth/

Tudo relacionado a autenticação e sessão. `AuthManager` cuida de gerar e conferir o hash da senha. `UserSession` guarda em memória qual usuário está logado no momento.

data/

As "entidades" do app (`User`, `Post`, `Comment`, `PostStats`), o acesso ao SQLite (`data/local/AppDatabase.kt`) e os DAOs (`data/dal/`), que são as únicas classes que executam SQL de verdade: `UserDAO`, `PostDAO`, `FollowDAO`, `LikeDAO` e `CommentDAO`.

navigation/

`Routes.kt` define cada tela do app como uma classe/objeto serializável. `AppNavigation.kt` monta o `NavHost` que liga cada rota à tela correspondente e decide pra onde navegar quando uma ação termina.

ui/screen/

As telas em si (um `@Composable` por arquivo): Login, Cadastro, Feed, Post (criar/editar), Perfil, Editar Perfil, Seguidores/Seguindo, Busca de Usuários e Detalhe do Post.

ui/component/

Pedaços de interface reaproveitados em mais de uma tela: `CardPost` (linha de um post no feed/perfil/detalhe), `UserRow` (linha de um usuário na busca e na lista de seguidores/seguindo) e `TopNavBar` (barra do topo).

ui/viewmodel/

A "ponte" entre a tela e o banco de dados: `AuthViewModel`, `PostViewModel`, `ProfileViewModel`, `FollowsViewModel`, `CommentViewModel`, `UsersViewModel`. Nenhuma tela acessa um DAO diretamente, sempre passa por um ViewModel.

ui/theme/

Cores, espaçamentos (`Dimens`) e tipografia do app. Centralizado aqui pra não espalhar número/cor mágica pelo código das telas.

validation/

Regras de "o que é um dado válido": `PostValidator` (limite de 280 caracteres) e `RegisterValidator` (regras de cadastro).

3. O padrão MVVM aplicado aqui

MVVM significa Model, View, ViewModel. É uma forma de separar "o que o app mostra" de "como o app funciona por dentro".

Camada	Onde fica	Responsabilidade
Model	`data/`	Dados puros (`Post`, `User`) e acesso ao banco (DAOs)
View	`ui/screen/`, `ui/component/`	Só desenha a tela, não sabe nada de SQL
ViewModel	`ui/viewmodel/`	Liga a View ao Model, guarda o estado da tela

A regra de ouro: a tela nunca fala com o banco direto. Ela só chama uma função do ViewModel (tipo `postViewModel.create(texto)`) e fica "olhando" um estado (`postViewModel.posts`) pra saber o que desenhar. Isso significa que dá pra trocar a tela inteira sem mexer na lógica de banco, e vice versa.

4. Fluxo completo de uma ação (exemplo: criar um post)

Passo a passo do que acontece quando alguém clica em "Postar":

1. `PostScreen` chama `postViewModel.create(texto)`.
2. `PostViewModel` pega o id do usuário logado em `UserSession`.

3. Valida o texto com `PostValidator.validate(...)`. Se for inválido, já para aqui e mostra erro.
4. Chama `PostDAO.create(...)` dentro de uma coroutine na thread de I/O (`Dispatchers.IO`), pra não travar a tela.
5. `PostDAO` monta um `ContentValues` e grava na tabela `posts` usando `insertOrThrow`.
6. Se der tudo certo, o ViewModel marca `published = true`.
7. A tela está "escutando" esse valor com um `LaunchedEffect`, percebe a mudança e navega de volta pro Feed.

> Esse mesmo padrão (tela chama função, ViewModel valida e grava, estado muda, tela reage) se repete em todas as outras operações: login, cadastro, editar post, seguir alguém, etc.

5. Banco de dados e migração automática

O banco é SQLite puro, sem Room nem nenhum ORM, acessado via `SQLiteOpenHelper` (classe `AppDatabase`). Cinco tabelas: `users` (com `cover_color`, a cor de capa do perfil, e os campos de bio/endereço: `bio`, `cep`, `rua`, `numero`, `bairro`, `cidade`, `estado`), `posts`, `follows`, `likes` e `comments`.

O schema inteiro e atualizado vive dentro do método `onCreate`, num lugar só (nunca espalhado em vários arquivos "update_1.sql", "update_2.sql" etc.). Quando uma tabela ou coluna nova é adicionada depois (follows na versão 2, cover_color/likes/comments na versão 3, bio/endereço na versão 4), o número da versão do banco sobe, e o método `onUpgrade` aplica a diferença sozinho, na hora que o app abre, sem precisar de nenhum passo manual.

```
override fun onUpgrade(db, oldVersion, newVersion) {
    if (oldVersion < 2) {
        db.execSQL("CREATE TABLE follows(...)")
    }
    if (oldVersion < 3) {
        db.execSQL("ALTER TABLE users ADD COLUMN cover_color ...")
        db.execSQL("CREATE TABLE likes(...)")
        db.execSQL("CREATE TABLE comments(...)")
    }
    if (oldVersion < 4) {
        db.execSQL("ALTER TABLE users ADD COLUMN bio ...")
        db.execSQL("ALTER TABLE users ADD COLUMN cep ...")
        // (e o resto dos campos de endereço, mesma ideia)
    }
}
```

Isso evita o problema clássico de "esqueceram de rodar a migração em produção": quem já tinha o app instalado numa versão antiga recebe a tabela/coluna nova automaticamente, sem perder os dados que já existiam.

6. Decisões de segurança

- **SQL Injection:** toda query usa `?` no lugar do valor (bind parameters). Nunca é feita concatenação de texto do usuário dentro de uma query.
- **Senha:** nunca é salva em texto puro. Vira um hash com PBKDF2 (60 mil iterações) mais um "salt" aleatório por usuário, então duas pessoas com a mesma senha nunca têm o mesmo hash salvo.
- **Dono do dado:** editar ou excluir um post só funciona se o `user_id` do post bater com quem está logado. Essa checagem acontece dentro do próprio `PostDAO`, não só na tela, então não dá pra burlar.
- **Banco fora do backup:** o arquivo do banco fica em `noBackupFilesDir`, pra não ser incluído em backup automático na nuvem do Android.

7. Mapa de telas e rotas

Rota	Tela	Precisa estar logado?
`LoginUser`	Login	Não
`RegisterUser`	Cadastro	Não
`Feed`	Lista de posts	Sim
`RegisterPost`	Criar post	Sim
`EditPost(postId)`	Editar post	Sim, só o dono
`DetailPost(postId)`	Detalhe do post	Sim
`Profile(userId)`	Perfil de um usuário	Sim
`EditProfile`	Editar o próprio perfil	Sim
`Follows(userId, type)`	Seguidores ou seguindo	Sim
`Users`	Busca de usuários	Sim

Quem tenta acessar uma rota protegida sem estar logado é redirecionado pro Login automaticamente (lógica dentro de um `LaunchedEffect` em `AppNavigation.kt`). As telas de Login e Cadastro são as únicas sem a barra de navegação do topo.

Documento gerado como material de apoio interno do projeto MySocial (trabalho acadêmico). Reflete o estado do código na branch de contingência do projeto.